

一、名词解释

1. 操作系统：管理计算机硬件和软件的计算机程序。
2. 操作系统：是控制和管理计算机系统的资源，并为用户提供接口的一组程序。
3. 分布式操作系统：是利用软件系统方式构建在计算机网络上的多种多处理机系统。
4. 实时操作系统：系统能及时响应外部事件的请求，在规定的时间内完成对事件的处理，并控制所有实时任务协调一致地运行。
5. 互斥共享：保证多个进程对这类资源进行互斥访问的一种机制。

三、填空题

1. 硬件系统 软件系统
2. 裸机
3. 硬件资源 软件资源
4. 顺序性
5. 共享性、异步性
6. 进程管理 进程调度
7. 程序系统接口
8. 单用户单任务 单用户多任务 多用户单任务

四、判断题

1. X
2. ✓
3. X
4. X
5. X
6. X
7. ✓
8. ✓
9. X
10. ✓

第二章.

一.

2. 直接制约: 由于进程间使用资源次序而产生的制约.
3. 临界区: 进程程序中, 涉及访问共享资源的程序段, 被称为临界区.
4. 进程同步: 指多个相关进程在执行次序上的协调.

三. 填空题.

1. 动态性 异步性
2. 用户态 用户模式

4. 系统空间

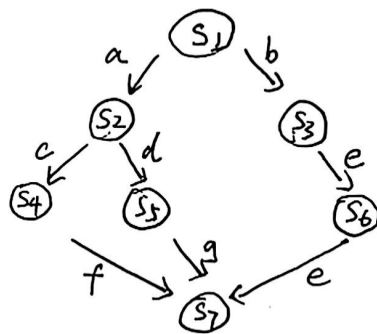
5. 动态 静态

6. 间接制约

四. 判断题

1. X 2. X 3. ✓ 4. X 5. ✓ 6. ✓ 7. X 8. X 9. ✓ 10. X

五. 2. 前驱关系: $S_1 \rightarrow S_2$ $S_1 \rightarrow S_3$ $S_2 \rightarrow S_4$ $S_2 \rightarrow S_5$ $S_3 \rightarrow S_6$ $S_4 \rightarrow S_7$ $S_5 \rightarrow S_7$ $S_6 \rightarrow S_7$



3. 为了描述上述同步^{问题}, 可定义如下的信号量:

semaphore empty=5, orange=0, apple=0, mutex=1;

爸爸、儿子和女儿的算法可描述为:

```
Dad() {  
    while (1) {  
        wait(empty);  
        wait(mutex);  
        将水果放入盘中;  
        signal(mutex);  
        if (放入的是桔子) signal(orange);  
        else signal(apple);  
    }  
}
```

华毓佳 2023013182

接五3:

```
Son() {  
    while(1) {  
        wait(orange);  
        wait(mutex);  
        从盘中取一个桔子;  
        signal(mutex);  
        signal(empty);  
        享用桔子;  
    }  
}
```

```
Daughter() {  
    while(1) {  
        wait(apple);  
        wait(mutex);  
        从盘中取一个苹果;  
        signal(mutex);  
        signal(empty);  
        享用苹果;  
    }  
}
```

4. 该文件打印过程的同步算法可描述为:

* semaphore empty 1=1, full 1=0, empty 2=1,
full 2=0;

```
PA() {  
    while(1) {  
        从磁盘读一个记录;  
        wait(empty 1);  
        将记录存放在缓冲区1中;  
        signal(full 1);  
    }  
}
```

```
PB() {  
    while(1) {  
        wait(full 1);  
        从缓冲区1中取出一个记录;  
        signal(empty 1);  
        wait(empty 2);  
        将记录复制到缓冲区2中;  
        signal(full 2);  
    }  
}
```

```
PC() {  
    while(1) {  
        wait(full 2);  
        从缓冲区2中取出一个记录;  
        signal(empty 2);  
        将取出的记录打印出来;  
    }  
}
```

```
}  
main() {  
    cobegin PA(); PB(); PC(); coend;  
}
```

```

5. int count = 0;
   semaphore mutex = 1, empty = 1, full = 0;
   semaphore payment = 0, receipt = 0;
   guest() {
       wait(mutex);
       if(count >= N) {
           signal(mutex);
           离开理发店;
       } else {
           count++;
           signal(mutex);
           在沙发中就座;
           wait(empty);
           离开沙发, 坐到理发椅上;
           wait(mutex);
           count--;
           signal(mutex);
           signal(full);
           理发;
           付费;
           signal(payment);
           wait(receipt);
           signal(empty);
           离开理发店;
       }
   }
}
Barber() {
    while(1) {
        wait(full);
        替顾客理发;
        wait(payment);
        收费;
        signal(receipt);
    }
}

```

第三章

完成的一个相对独立的用户任务。

- 一、1. 作业是用户向计算机提交的一个计算任务，它由系列的程序、数据和作业说明书组成是用户要求操作系统完成的一个相对独立的用户任务。
2. 处理机调度：在多道程序环境下，内存中存在着多个进程，其数目往往多于处理机数目，要求系统能按某种算法，动态地将处理机分配给处于就绪状态的一个进程，使之执行。
3. 周转时间是指从作业提交到作业完成的时间间隔，即作业的完成时间减去作业的提交时间。
4. 死锁是指两个或多个进程在执行过程中，因争夺资源而造成的一种互相等待的现象。
5. 临时性资源是指在进程执行过程中临时申请和释放的资源，如内存空间、I/O设备等。

三.

2. 后备运行 完成、

6. 预防死锁、避免死锁 检测死锁、解除死锁。

四. X √ X X √ √ X √ X √

五. 1. FCFS. 调度顺序 $A \rightarrow B \rightarrow C \rightarrow D \rightarrow E$

① 完成时间: $A: 3 \quad B: 3+6=9 \quad C: 9+4=13 \quad D: 13+5=18 \quad E: 18+2=20$

② 周转时间: $A: 3-0=3 \quad B: 9-2=7 \quad C: 13-4=9 \quad D: 18-5=13 \quad E: 20-8=12$

③ 带权周转时间: $A: 3/3=1 \quad B: 7/6=1.167 \quad C: 9/4=2.25 \quad D: 12/5=2.4 \quad E: 12/2=6$

④ 平均周转时间: $(3+7+9+12+12)/5=8.6$

⑤ 平均带权周转时间: $(1+1.167+2.25+2.4+6)/5=2.3634$

SJF

$A \rightarrow B \rightarrow E \rightarrow C \rightarrow D$

① $A: 3 \quad B: 6 \quad E: 3 \quad C: 4 \quad D: 5$

② $A: 3 \quad B: 7 \quad E: 3 \quad C: 11 \quad D: 14$

③ $A: 3/3=1 \quad B: 7/6=1.167 \quad E: 3/2=1.5 \quad C: 11/4=2.75 \quad D: 14/5=2.8$

④ $(3+7+3+11+14)/5=7.6$

⑤ $(1+1.167+1.5+2.75+2.8)/5=1.8434$

HRRN

$A \rightarrow B \rightarrow C \rightarrow E \rightarrow D$

① $A: 3 \quad B: 9 \quad C: 13 \quad E: 15 \quad D: 20$

② $A: 3-0=3 \quad B: 9-2=7 \quad C: 13-4=9 \quad E: 15-8=7 \quad D: 20-6=14$

③ $A: 3/3=1 \quad B: 7/6=1.167 \quad C: 9/4=2.25 \quad E: 14/5=2.8 \quad D: 7/2=3.5$

④ $(3+7+9+7+14)/5=8$

⑤ $(1+1.167+2.25+2.8+3.5)/5=2.1434$

2. 任务A的松弛度 = $20ms - 0ms - 10ms = 10ms$

任务B的松弛度 = $50ms - 0ms - 10ms = 40ms$

任务C的松弛度 = $50ms - 0ms - 15ms = 35ms$

首先执行任务A, 10ms后, A执行完毕, 重新计算松弛度.

任务B的松弛度 = $50ms - 10ms - 10ms = 30ms$

任务C的松弛度 = $50ms - 10ms - 15ms = 25ms$.

接下来执行任务C, 25ms后, C执行完毕, 再次计算.

A: $20ms - 20ms - 10ms = -10ms$.

B: $50ms - 35ms - 10ms = 5ms$

因此执行任务B.

综上, 调度顺序为: 任务A $\rightarrow$ 任务C $\rightarrow$ 任务B.

3. (1) $Allocation = Max - Need$.

Process	Allocation
P ₀	0 0 3 2
P ₁	1 0 0 0
P ₂	1 3 5 4
P ₃	0 3 3 2
P ₄	0 0 1 4

找到安全序列 {P₀, P₃, P₄, P₁, P₂}. 故系统安全.

安全性过程检查:

进程 \ 资源情况	Work				Need				Allocation				Work+Allocation				Finish
	A	B	C	D	A	B	C	D	A	B	C	D	A	B	C	D	
P ₀	1	6	2	2	0	0	1	2	0	0	3	2	1	6	5	4	True
P ₃	1	6	5	4	0	6	5	2	0	3	3	2	1	9	8	6	True
P ₄	1	9	8	6	0	6	5	6	0	0	1	4	1	9	9	10	True
P ₁	1	9	9	10	1	7	5	0	1	0	0	0	2	9	9	10	True
P ₂	2	9	9	10	2	3	5	6	1	3	5	4	3	12	14	14	True

(2) P₂发出请求向量 $Request(1, 2, 2, 2)$.

① $Request_2(1, 2, 2, 2) \leq Need_2(2, 3, 5, 6)$

② $Request_2(1, 2, 2, 2) \leq Available(1, 6, 2, 2)$

③ 系统假定可为P₂分配已资源, 并修改Available, Allocation₂和Need₂向量.

$Available = (0, 4, 0, 0)$

$Allocation_2 = (2, 5, 7, 6)$

$Need_2 = (1, 1, 3, 4)$

④ 此时对所有的进程, 条件 $Need_i \leq Available(0, 4, 0, 0)$ 都不成立, 系统进入不安全状态. 因此系统不能把资源分配给它.

(3) 不会. 因为此时上述进程并没有申请新的资源, 并因得不到资源而进入阻塞状态. 当上述进程提出新的请求, 所有没执行完的多个进程因得不到资源而阻塞并形成循环等待链时, 系统才进入死锁状态.

RR $A \rightarrow B \rightarrow C \rightarrow D \rightarrow E$

① $A: 4 \quad B: 19 \quad C: 16 \quad D: 20 \quad E: 13$

② $A: 4 \quad B: 19-2=17 \quad C: 16-4=12 \quad D: 20-6=14 \quad E: 13-8=5$

③ $A: 4 \div 3 = 1.33 \quad B: 17 \div 6 = 2.83 \quad C: 12 \div 4 = 3 \quad D: 14 \div 5 = 2.8 \quad E: 5 \div 2 = 2.5$

④ $(4+17+12+14+5) \div 5 = 10.4$

⑤ $(1.33+2.83+3+2.8+2.5) \div 5 = 2.492$